Chapter 5 — Effects and Refs

So far every component has been a pure function of props and state (Chapter 2 §2.5), and every change has come from an event handler (Chapter 3). Real apps also need to talk to things **outside React**: a chat server connection, a timer, a browser API (a built-in function or object the browser offers to code) such as `document.title`, a **third-party** (written by someone else, not by you or React) map widget, a DOM node that must be focused or measured. React's two **escape hatches** (deliberate ways to step outside React's normal render-from-state model) for this are **Effects** (`useEffect`, "run this code after rendering to synchronise with something outside") and **refs** (`useRef`, "remember a value, or hold a DOM node, without re-rendering").

They are also the most over-used Hooks (React's built-in `use…` functions). Half of this chapter is about **when not to use an Effect**, because unnecessary Effects make apps slower and buggier.

What you'll learn

#	Topic	You will be able to…
5.1	Where side effects belong	choose between render, event handler and Effect
5.2	`useEffect` basics	write an Effect and predict when it runs
5.3	Cleanup	undo subscriptions, connections, timers and listeners
5.4	StrictMode's double run	understand setup → cleanup → setup in development and why it's your friend
5.5	Dependencies in depth	satisfy the linter by changing code; use updaters and `useEffectEvent`
5.6	You might not need an Effect	replace Effects with rendering, `key`, handlers and better state
5.7	Fetching data in Effects	avoid race conditions with an ignore flag or `AbortController`; know the limits
5.8	`useRef` for values	keep mutable values that don't affect rendering
5.9	Refs to DOM nodes	focus, scroll, measure; ref callbacks and lists of refs
5.10	Passing refs to components	`ref` as a prop (React 19), `forwardRef` (legacy), `useImperativeHandle`
5.11	`useLayoutEffect`	measure before the browser paints; know its cost
5.12	Putting it together	a city search with five Effects and three refs

Code: `code/` — React 19.3 in jsdom (a browser-like DOM that runs inside Node.js). jsdom has no **layout engine** (the part of a browser that works out each element's size and position), so the examples that measure or scroll fake the few layout numbers they need and say so.

5.1 Where side effects belong

What it is

A **side effect** is anything a piece of code does besides computing a return value: changing the DOM by hand, sending a request, starting a timer, writing to `localStorage` (small key-value storage kept by the browser), logging analytics (usage statistics sent to a tracking service). React gives you three places to put code:

Place	Runs when	Must be	Use it for
Rendering (the component body)	Every time React needs the UI	**Pure** — same inputs, same JSX, no side effects	Computing JSX and derived values
Event handlers	When a specific user action happens	Anything goes	Side effects **caused by a particular interaction**: submit a form, buy a product, start a timer on click
Effects (`useEffect`)	**After** a render is committed (React has applied its changes to the real DOM)	Should be undoable (cleanup)	Side effects **caused by rendering itself** — keeping the component **synchronised** with an **external system** (anything React doesn't control: network, timers, browser APIs, other libraries) while it's on screen

The question to ask is **"why does this code run?"**

- *Because the user clicked "Send"* → event handler.
- *Because this component is visible and must stay connected / subscribed / in sync* → Effect.
- *Because I need a value computed from props or state* → neither: compute it during render.

Analogy: a shop assistant (your component). Answering a customer's question is rendering. Ringing up a sale because a customer asked to pay is an event handler. Keeping the shop lights on **while the shop is open** — and switching them off at closing time — is an Effect with cleanup.

TL;DR

Pure calculations → render. Caused by a user action → handler. Needed because the component is on screen, to sync with something outside React → Effect.

5.2 `useEffect` basics

What it is

```
useEffect(setup, dependencies?)
// setup: function run after the render is committed; may return a cleanup function
// dependencies: array of every reactive value setup reads (props, state, values derived from them)
```

An **Effect** runs **after** React has updated the DOM (and normally after the browser has **painted** — drawn the pixels on screen), so it never blocks the user from seeing the new screen. A **reactive value** is anything declared in the component body that can change between renders: props, state, and variables computed from them.

The dependency array controls re-runs:

You write	Setup runs
`useEffect(fn)` — no array	After **every** render
`useEffect(fn, [])` — empty array	Once after the first render (**mount** = first appears); cleanup on **unmount** (= removed from the screen)
`useEffect(fn, [a, b])`	After mount, and after any render where `a` or `b` changed (compared with `Object.is`)

How it works

`code/01_effect_basics.jsx`:

```
function Demo() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState('Ada');
  console.log(`render      count=${count} name=${name} | DOM says: ${document.querySelector('h1')?.textContent ??
'(nothing yet)'}`);

  useEffect(() => {
    console.log(`  effect A (no array)   -> after EVERY render; DOM says:
${document.querySelector('h1').textContent}`);
  });
  useEffect(() => {
    console.log('  effect B ([])         -> once, after the first render (mount)');
    return () => console.log('  cleanup B            -> on unmount');
  }, []);
  useEffect(() => {
    console.log(`  effect C ([name])     -> name is now ${name}`);
    return () => console.log(`  cleanup C            -> cleans up the "${name}" effect`);
  }, [name]);

  return (
    <>
      <h1>{name}: {count}</h1>
      <button onClick={() => setCount((c) => c + 1)}>count</button>
      <button onClick={() => setName('Grace')}>name</button>
    </>
  );
}
```

Output:

```
--- mount
render      count=0 name=Ada | DOM says: (nothing yet)
  effect A (no array)   -> after EVERY render; DOM says: Ada: 0
  effect B ([])         -> once, after the first render (mount)
  effect C ([name])     -> name is now Ada
--- click count (name unchanged)
render      count=1 name=Ada | DOM says: Ada: 0
  effect A (no array)   -> after EVERY render; DOM says: Ada: 1
--- click name
render      count=1 name=Grace | DOM says: Ada: 1
  cleanup C            -> cleans up the "Ada" effect
  effect A (no array)   -> after EVERY render; DOM says: Grace: 1
  effect C ([name])     -> name is now Grace
--- click name again (same value: nothing committed, so no effects)
render      count=1 name=Grace | DOM says: Grace: 1
--- unmount
  cleanup B            -> on unmount
  cleanup C            -> cleans up the "Grace" effect
```

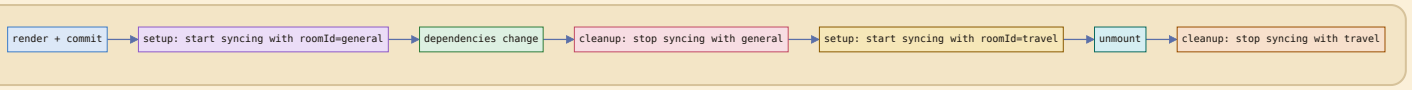
What to notice:

1. **Effects run after the DOM is updated.** During render the `<h1>` still shows the old text; inside the Effect it shows the new text.
2. **Dependencies are compared with the previous render.** Clicking "count" didn't re-run C (name unchanged); clicking "name" did.
3. **Cleanup runs before the next setup** — and it sees the *old* values (`"Ada"`), because each render's Effect closes over that render's snapshot (a **closure** — a function remembers the variables that existed where it was created; Chapter 3 §3.3).
4. **Same state value → nothing is committed.** React may call the component once more to check (the extra "render" line) but doesn't update the DOM or run Effects.
5. **Unmount** runs every remaining cleanup.

An Effect's life, from its own point of view

Don't think in "mount/update/unmount" (class-component lifecycles). Think: **"start synchronising with X; stop synchronising with X."** React may start and stop as many times as needed — every time the

dependencies change, and once extra in development (§5.4). If your setup/cleanup pair is symmetrical, how often it runs doesn't matter.



Rules

- Call `useEffect` at the **top level** of the component (Hook rules — Chapter 7).
- The setup function itself must not be `async` (it would return a **promise** — an object standing for a result that arrives later — not a cleanup). Define an async function inside and call it (§5.7).
- Effects **don't run on the server** (server rendering — producing HTML on the server before the page reaches the browser; Chapter 13).

In the interview

30-second answer: "`useEffect` synchronises a component with an external system. The setup runs after the render is committed; the optional cleanup runs before the next setup and on unmount. The dependency array lists every reactive value the Effect reads: no array runs after every render, an empty array only on mount, and otherwise when a dependency changes by `Object.is`. I think of it as start-sync/stop-sync rather than lifecycle events."

- *Does `useEffect` run before or after paint?* — Normally after the browser paints; `useLayoutEffect` runs before (§5.11).
- *Why can't the setup be `async`?* — It must return nothing or a cleanup function, not a promise.

TL;DR

`useEffect(() => { start; return () => stop; }, [deps])`. Runs after commit; re-runs when deps change; cleanup before the next run and on unmount.

5.3 Cleanup — undo what setup did

What it is

The **cleanup function** is what your setup returns. It must **undo** the setup: disconnect what you connected, unsubscribe what you subscribed, clear the timer you started, remove the listener you added, cancel the request you sent. Without it, each re-run *adds* another connection/timer/listener — a **leak** (resources that are never released and pile up, wasting memory and network).

How it works

`code/02_cleanup_and_strict_mode.jsx`:

```jsx
function ChatRoom({ roomId }) {
  useEffect(() => {
    const connection = createConnection(roomId);   // setup
    connection.connect();
    return () => connection.disconnect();          // cleanup
  }, [roomId]);
  return <h2>Welcome to {roomId}</h2>;
}

function LeakyChatRoom({ roomId }) {
  useEffect(() => {
    createConnection(roomId).connect();            // no cleanup!
  }, [roomId]);
  return <h2>Leaky {roomId}</h2>;
}
// rendered inside <StrictMode>; the user switches room, then closes the chat

// Timers and listeners follow the same shape.
function Clock() {
  const [ticks, setTicks] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setTicks((t) => t + 1), 10);
    const onKey = (e) => console.log('  key pressed:', e.key);
    window.addEventListener('keydown', onKey);
    return () => { clearInterval(id); window.removeEventListener('keydown', onKey); };
  }, []);
  return <p>ticks: {ticks}</p>;
}
```

Output:

```
=== WITH cleanup, inside <StrictMode> (development)
mount:
  connect    general (open: 1)
  disconnect general (open: 0)
  connect    general (open: 1)
switch room:
  disconnect general (open: 0)
  connect    travel (open: 1)
close:
  disconnect travel (open: 0)
connections still open: 0

=== WITHOUT cleanup, inside <StrictMode> (development)
mount:
  connect    general (open: 1)
  connect    general (open: 2)
switch room:
  connect    travel (open: 3)
close:
connections still open: 3

  key pressed: Escape
clock shows ticks: 5 (one interval, one listener — the StrictMode duplicate was cleaned up)
after unmount the listener is gone (no "key pressed: Enter" above)
```

With cleanup, there is always **exactly one** open connection, to the right room. Without it, connections pile up — to rooms the user has already left — and keep running after the component is gone.

Common setup/cleanup pairs

Setup	Cleanup
`connection.connect()`	`connection.disconnect()`
`const id = setInterval(…)` / `setTimeout(…)`	`clearInterval(id)` / `clearTimeout(id)`
`window.addEventListener('resize', fn)`	`window.removeEventListener('resize', fn)` — the **same** function reference
`const unsub = store.subscribe(fn)`	`unsub()`
`const controller = new AbortController(); fetch(url, { signal })`	`controller.abort()`
`const obs = new IntersectionObserver(…); obs.observe(el)` (reports when an element scrolls into view)	`obs.disconnect()`
`const map = new MapWidget(el)`	`map.destroy()`
`dialog.showModal()`	`dialog.close()`
Start an animation	Reset it to the initial state

Common mistakes

- Removing a listener with a *new* arrow function (`removeEventListener('x', () => fn())`) — nothing is removed. Keep the function in a variable.
- Forgetting that a cleanup sees the old render's values — that's usually what you want (disconnect the *old* room).
- Writing cleanup for something that can't be undone (a one-off POST request) — that code probably belongs in an event handler (§5.6).

TL;DR

Return a function that exactly undoes the setup. Then re-runs, room switches and unmounts never leak.

5.4 StrictMode's double run (development only)

What it is

When your app is wrapped in `<StrictMode>` (Vite and most templates do this — Chapter 1 §1.6), React in **development** mounts every component, then **immediately simulates an unmount and a re-mount**: setup → cleanup → setup. In the output above, that's the `connect / disconnect / connect` on mount. Production builds (the optimised version real users get) run the setup once.

React 19 applies the same check to **ref callbacks** (§5.9) in development, and React 19.3 also does it for components that were server-rendered and then **hydrated** (React attached itself to HTML the server already produced).

Why React does this

Users *do* unmount and remount components: they navigate away and back, a tab is hidden with `<Activity>` (React 19.2's component for hiding a subtree while keeping its state) and shown again, a list is filtered. If setup and cleanup are not symmetrical, those real situations produce bugs that are hard to reproduce. StrictMode forces the situation on every mount in development so you see the bug immediately — the leaky version shows `open: 2` right at mount.

How to handle it

The fix is **never** "stop React running it twice". The fix is a correct cleanup:

Symptom in development	Wrong fix	Right fix
Two connections / subscriptions	A ref flag "already connected"	Return `disconnect` / `unsubscribe`
Two intervals ticking twice as fast	Disable StrictMode	`return () => clearInterval(id)`
Two fetches in the Network tab (the browser DevTools panel listing every request)	Ref flag	Harmless if you ignore/abort the first (§5.7); or use a data library with caching
Analytics event logged twice	Ref flag	Acceptable in dev (production logs once), or send analytics from the event that caused it
Code that must run once per **app load** (initialising a SDK — Software Development Kit, a vendor's client library)	Effect with a ref	Top-level module code, or a module-level `let didInit = false` guard

In the interview

> **30-second answer:** "In development, StrictMode runs every Effect's setup, cleanup and setup again on mount to check the cleanup mirrors the setup. It doesn't happen in production. If the double run causes a visible problem, the Effect is missing a cleanup — the fix is to add one, not to suppress the second run."

TL;DR

Dev-only setup → cleanup → setup exposes missing cleanups. Fix the cleanup; don't fight the double run.

5.5 Dependencies in depth

The rule

Every **reactive value** the Effect reads must be in its dependency array. The `react-hooks/exhaustive-deps` lint rule (a check run by a **linter** — a tool such as ESLint that reads your code and flags likely bugs; this rule lives in `eslint-plugin-react-hooks`, included by Vite's React template) checks this for you. Values that never change — module-level constants, `useRef` objects, state setters, the `dispatch` from `useReducer`/`useActionState` — can be left out.

You don't choose the dependencies; the code does. If an Effect re-runs too often, change the code so it reads fewer reactive values. Never silence the linter with `// eslint-disable-next-line` — it leads to Effects reading stale values.

Techniques

`code/03_dependencies.jsx` shows three:

```jsx
// 1. An object created during render is a NEW object every render -> Effect re-runs on every keystroke.
function ObjectDep({ roomId }) {
  const [draft, setDraft] = useState('');
  const options = { serverUrl: 'https://chat.example', roomId };     // new {} each render
  useEffect(() => connect(options), [options]);
  // ... controlled <input> for draft
}
// Fix: create the object INSIDE the Effect and depend on the primitive.
function PrimitiveDep({ roomId }) {
  const [draft, setDraft] = useState('');
  useEffect(() => {
    const options = { serverUrl: 'https://chat.example', roomId };
    return connect(options);
  }, [roomId]);
  // ...
}

// 2. Reading state only to compute the next state: use an updater, and the dependency disappears.
useEffect(() => {
  const id = setInterval(() => (useUpdater ? setCount((c) => c + 1) : setCount(count + 1)), 10);
  return () => clearInterval(id);
}, useUpdater ? [] : [count]);

// 3. useEffectEvent (React 19.2+): read the LATEST props/state inside an Effect without making them dependencies.
function ChatRoom({ roomId, theme }) {
  const onConnected = useEffectEvent(() => log.push(`connected to ${roomId}, toast in ${theme} theme`));
  useEffect(() => {
    log.push(`connect ${roomId}`);
    const t = setTimeout(() => onConnected(), 5);                // "the server said connected"
    return () => { clearTimeout(t); log.push(`disconnect ${roomId}`); };
  }, [roomId]);                                                   // theme is NOT a dependency
  return null;
}
// theme changes light -> dark while connecting; later roomId changes to travel
```

Output (the counts in the middle two lines depend on timer timing):

```
object dependency   : connects after typing 5 letters = 6
primitive dependency: connects after typing 5 letters = 1
count dep: count=5 interval restarts=6
updater: count=5 interval restarts=1
useEffectEvent: [
  'connect general',
  'connected to general, toast in dark theme',
  'disconnect general',
  'connect travel',
  'connected to travel, toast in dark theme'
]
```

1. **Objects/functions created during render** are new on every render, so `Object.is` always says "changed": 6 connections for 5 keystrokes. Move them **inside** the Effect (or outside the component if they don't use props/state) and depend on **primitives** (simple values — strings, numbers, booleans — compared by value, not by reference) (`roomId`).
2. **Updater functions** (Chapter 3 §3.5) let an interval increment `count` without reading it, so the interval is created once instead of being torn down and restarted on every tick.
3. `useEffectEvent` splits logic in two: the *reactive* part (connect to `roomId` — should re-run when it changes) and the *non-reactive* part (show a **toast** — a small pop-up notification that disappears by itself — in the *current* `theme` — should see the latest value but must **not** reconnect when the theme changes). Changing the theme did not reconnect, yet the toast used "dark". Changing the room did reconnect.

`useEffectEvent` rules (stable since React 19.2)

- Call the returned function **only from inside Effects** (or other Effect Events), never during render or from event handlers.
- **Don't** list it in dependencies (the linter knows it's non-reactive).

- Declare it in the same component/Hook as the Effect that uses it; don't pass it to other components.
- Use it for genuinely event-like logic ("connected", "a message arrived", "log a visit with the current cart size"), not as a trick to hide real dependencies.

Other techniques

Problem	Technique
Function prop from parent re-creates every render	Call it from an Effect Event, or have the parent memoise it (keep the same function between renders) with `useCallback` (Chapter 10)
Effect reads a whole object prop but needs one field	Destructure the primitive outside the Effect: `const { id } = user;` → `[id]`
Two unrelated things in one Effect	Split into two Effects, each with its own dependencies
Mutable value such as `location.pathname` or a ref's `.current`	Not a valid dependency (changes don't re-render). Subscribe with `useSyncExternalStore` (a Hook for reading a value that lives outside React and re-rendering when it changes; Chapter 7)

Self-check

1. Why did `ObjectDep` reconnect on every keystroke? → A new `options` object each render; `Object.is` compares references.
2. How can an interval Effect avoid depending on `count`? → `setCount(c => c + 1)`.
3. What is `useEffectEvent` for? → Reading latest values in an Effect without making them re-run triggers.

TL;DR

List everything the Effect reads; reduce the list by changing code — move objects inside, use updaters, split Effects, and use `useEffectEvent` for non-reactive logic. Never lie to the linter.

5.6 You might not need an Effect

The problem it solves

Beginners reach for `useEffect` whenever "something should happen when X changes". Most of those cases involve **no external system** and are better written without an Effect. Unneeded Effects cost an **extra render**, can show **stale UI for a frame** (one screen refresh, about 16 ms at 60 frames per second), and create **chains** that are hard to follow.

How to recognise it

If the Effect only (a) calls `setState` based on props/state, or (b) does something because of a *user action* that just happened — it's probably not needed.

`code/04_you_might_not_need_an_effect.jsx` runs five before/after pairs:

```jsx
// 1. Derived data: computing in an Effect = one extra render + one stale frame.
function FullNameEffect({ first, last }) {
  const [fullName, setFullName] = useState('');
  useEffect(() => setFullName(`${first} ${last}`), [first, last]);   // 🔴
  return <p id="n">{fullName}</p>;
}
function FullNameRender({ first, last }) {
  const fullName = `${first} ${last}`;                               // ✅ just compute it
  return <p id="n">{fullName}</p>;
}

// 2. Resetting state when a prop changes: Effect shows the OLD comment for one frame; a key doesn't.
function CommentBoxEffect({ userId }) {
  const [comment, setComment] = useState('');
  useEffect(() => setComment(''), [userId]);                         // 🔴
  // ...
}
function Profile({ userId }) {
  return <CommentBox key={userId} userId={userId} />;                // ✅ new key = fresh state
}

// 3. Event-specific logic belongs in the event handler, not in an Effect watching state.
function ProductEffect({ initialInCart }) {
  const [inCart, setInCart] = useState(initialInCart);
  useEffect(() => { if (inCart) toasts.push('effect: "Added to cart!"'); }, [inCart]);      // 🔴 also fires on page load
  return <button onClick={() => setInCart(true)}>Buy</button>;
}
function ProductHandler({ initialInCart }) {
  const [inCart, setInCart] = useState(initialInCart);
  function handleBuy() { setInCart(true); toasts.push('handler: "Added to cart!"'); }      // ✅ only when the user buys
  return <button onClick={handleBuy}>Buy</button>;
}

// 4. Notifying the parent: from an Effect = a second render pass; from the handler = one pass.
function Toggle({ onChange, viaEffect }) {
  const [on, setOn] = useState(false);
  useEffect(() => { if (viaEffect) onChange(on); }, [on]);           // 🔴 (and runs on mount too)
  function handleClick() { const next = !on; setOn(next); if (!viaEffect) onChange(next); }   // ✅ same event
  return <button onClick={handleClick}>{on ? 'On' : 'Off'}</button>;
}

// 5. Adjusting SOME state when a prop changes: best is to derive; else set state during render (no Effect).
function ListAdjust({ items }) {
  const [selection, setSelection] = useState(items[0]);
  const [prevItems, setPrevItems] = useState(items);
  if (items !== prevItems) {            // prop changed since last render -> fix up state right now
    setPrevItems(items);
    setSelection(null);                 // React re-runs this component immediately, before touching the DOM
  }
  return <p id="s">{selection ?? 'none'}</p>;
}
function ListDerived({ items }) {
  const [selectedId, setSelectedId] = useState('a');
  const selection = items.find((i) => i === selectedId) ?? null;     // ✅ derive: kept if still present
  return <p id="s">{selection ?? 'none'}</p>;
}
```

Output ("screen after each commit" is recorded right after every DOM update):

```
1 derived via Effect : renders=4 | screen after each commit: ["","Ada Lovelace","Ada Lovelace","Grace Hopper"]
1 derived via render : renders=2 | screen after each commit: ["Ada Lovelace","Grace Hopper"]
2 reset via Effect: screen after switching user: ["hi Ada","(empty)"]
2 reset via key   : screen after switching user: ["(empty)"]
3 ProductEffect : toasts on page load = ["effect: \"Added to cart!\""]
3 ProductHandler: toasts on page load = []
4 notify via Effect : component renders for one click = 3 | parent shows on
4 notify via handler: component renders for one click = 2 | parent shows on
5 ListAdjust : renders=3 | selection after items changed = none
5 ListDerived: renders=2 | selection after items changed = a
```

Reading the evidence:

1. The Effect version rendered **twice as often** and committed an **empty name** first, then a **stale "Ada Lovelace"** while the props already said Grace Hopper.
2. The Effect reset committed the previous user's draft ("hi Ada") to the screen before clearing it — on a chat app that's a privacy bug. The `key` version never shows it.
3. The Effect "Added to cart!" fired when a page simply **loaded** with an item already in the cart. The handler version fires only when the user buys.
4. Notifying the parent from an Effect costs an extra render pass (and fires on mount). Calling `onChange` in the same handler batches both updates into one render.
5. Setting state during render (with a "previous value" state) avoids the Effect, but deriving from an ID is simpler still — and keeps the selection when the item survives.

The full catalogue

You want to…	Instead of an Effect…
Compute something from props/state	Compute during render (Chapter 3 §3.8)
Cache an expensive calculation	`useMemo` (Chapter 10)
Reset **all** state when a prop changes	Give the component a `key`
Adjust **some** state when a prop changes	Derive it; if you must, set state during render with a stored previous value
Share logic between two event handlers	Extract a function and call it from both
Send a request because the user did something (POST on submit)	Do it in the event handler / form Action (Chapter 4)
Run a chain of state updates (A changes → set B → set C…)	Calculate all next state in the one handler
Initialise the app once (read a token — e.g. a login credential string, init a SDK)	Top-level module code or a `didInit` module flag
Tell the parent about a change	Call the parent's callback in the same handler, or lift the state up (Chapter 3 §3.7)
Pass fetched data up to the parent	Fetch in the parent and pass it down
Subscribe to an external store	`useSyncExternalStore` (Chapter 7)
Fetch data	An Effect *with cleanup* is acceptable (§5.7), but prefer a framework loader (a function the router runs to fetch a page's data before showing it) or TanStack Query (a data-fetching and caching library) (Chapter 6 §6.9, Chapter 9)

What *does* need an Effect

Connecting to a server or socket (a WebSocket — a connection kept open so the server can push messages), subscribing to browser events (`resize`, `online`, keyboard shortcuts), timers that run while a component is visible, synchronising with a non-React widget (a map, a video player, `dialog.showModal()`), updating `document.title`, logging an **impression** (an analytics record that something was seen) **because the component was shown**, and fetching (with cleanup).

In the interview

> **30-second answer:** "Effects are for synchronising with external systems. If there's no external system — just transforming props into state or reacting to a user event — I don't use one. Derived values are computed during render, expensive ones memoised; full resets use a `key`; event-caused logic goes in the handler; parents are notified in the same handler. Unneeded Effects add renders, show stale frames, and chain into hard-to-follow code."

- *How do you reset a form when `userId` changes?* — `<Form key={userId} />`.
- *Where does 'POST analytics when the form is submitted' go?* — In the submit handler. 'Log that the page was viewed' goes in an Effect.

Self-check

1. `useEffect(() => setTotal(a + b), [a, b])` — better? → `const total = a + b;` in render.
2. Why is the Effect reset of a comment box a privacy risk? → The old user's text is committed to the screen for a frame.
3. Name three legitimate Effects. → Socket connection, window listener, `document.title` sync (also timers, widgets, fetching with cleanup).

TL;DR

No external system → no Effect. Compute in render, reset with `key`, act in handlers, notify parents in the same event, subscribe with `useSyncExternalStore`.

5.7 Fetching data in Effects and race conditions

What it is

Fetching in an Effect means: when the component shows with some input (a person, a search query), request the matching data and put it in state. It is a legitimate Effect — the component is synchronising with a server — but it has a classic trap: a **race condition** (a bug where the result depends on which of several overlapping operations finishes first), where two requests are **in flight** (sent but not yet answered) and the **slower, older** one finishes last and overwrites the correct result.

How it works — three versions

`code/05_fetch_race_conditions.jsx` (the fake API answers Alice in 60 ms and Bob in 10 ms; the user picks Alice, then quickly Bob):

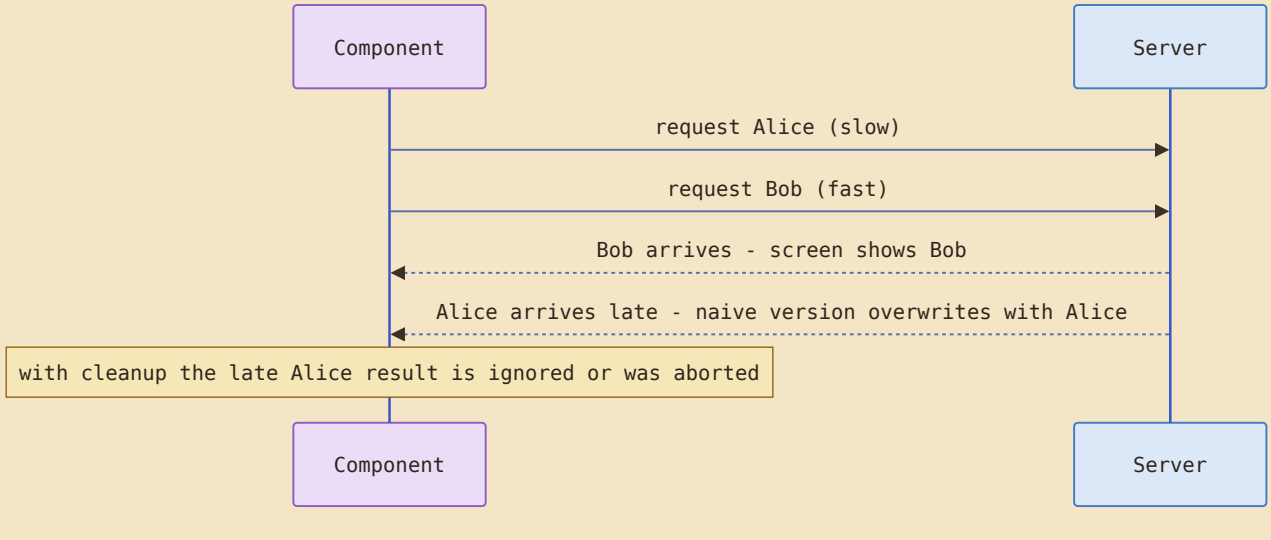
```jsx
function BioNaive({ person }) {                              // 🔴 no cleanup
  const [bio, setBio] = useState(null);
  useEffect(() => { fetchBio(person).then(setBio); }, [person]);
  return <p>{bio ?? 'Loading…'}</p>;
}

function BioIgnore({ person }) {                             // ✅ ignore stale responses
  const [bio, setBio] = useState(null);
  useEffect(() => {
    let ignore = false;
    setBio(null);
    fetchBio(person).then((result) => { if (!ignore) setBio(result); });
    return () => { ignore = true; };                         // runs when person changes or on unmount
  }, [person]);
  return <p>{bio ?? 'Loading…'}</p>;
}

function BioAbort({ person }) {                              // ✅ cancel the request itself + loading/error
states
  const [state, setState] = useState({ status: 'loading' });
  useEffect(() => {
    const controller = new AbortController();
    setState({ status: 'loading' });
    fetchBio(person, { signal: controller.signal })
      .then((bio) => setState({ status: 'success', bio }))
      .catch((err) => { if (err.name !== 'AbortError') setState({ status: 'error', message: err.message }); });
    return () => controller.abort();
  }, [person]);
  if (state.status === 'loading') return <p>Loading…</p>;
  if (state.status === 'error') return <p role="alert">Could not load: {state.message}</p>;
  return <p>{state.bio}</p>;
}
```

Output:

```
BioNaive  selected Bob, screen shows: "This is Alice's bio."
BioIgnore selected Bob, screen shows: "This is Bob's bio."
BioAbort  selected Bob, screen shows: "This is Bob's bio." [ 'aborted Alice' ]
BioAbort  error state: Could not load: 404 Not Found
```



- **Naive**: the user selected Bob but sees Alice — the late response won.
- **Ignore flag**: each Effect run has its own `ignore` variable; the cleanup of the Alice run sets *its* flag, so Alice's result is dropped.
- **`AbortController`** (a browser object whose `abort()` cancels the requests given its `signal`): actually cancels the old request (saves **bandwidth** — the amount of data sent over the network; `fetch` rejects

with an `AbortError`, which you must not show as an error). Also models the three states you always need: **loading, error, success**.

With real `fetch`, check `response.ok` yourself — `fetch` only rejects on network failure, not on HTTP 404/500 (HTTP **status codes**: 404 = not found, 500 = server error; see System Design Fundamentals for HTTP basics):

```
// (pattern — not run)
const res = await fetch(`/api/bio/${person}`, { signal });
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const data = await res.json();
```

Why fetching in Effects isn't the end of the story

Downside	Consequence
Starts only after render + paint	Slower than fetching during navigation or on the server
Parent fetches, then child renders and fetches	**Network waterfalls** (requests that wait for each other one after another instead of running in parallel)
No cache	Going back and forth re-fetches everything
Boilerplate (repetitive set-up code)	Loading/error/abort/retry in every component
No deduplication (merging identical requests into one)	Two components needing the same data make two requests

That's why real apps use a **framework's data loading** (React Router loaders, Next.js Server Components — Chapters 9, 13) or a **client cache** (a store of already-fetched data kept in the browser) such as **TanStack Query** (Chapter 6 §6.9), which handle caching, deduplication, retries, and races for you. React 19's `use()` with Suspense (React's mechanism for showing a fallback such as a spinner while something is loading) is another option for promises created outside render (Chapter 9).

Common mistakes

- `useEffect(async () => …)` — setup must not return a promise.
- No cleanup → race conditions and "state update on unmounted component" style bugs.
- Showing "Error: AbortError" after a cancellation — filter it out.
- Putting the fetched data *and* a copy of it filtered into state — keep the data, derive the filtered list.

In the interview

30-second answer: "Fetching in an Effect is fine if you handle races: either an `ignore` flag set in cleanup, or an `AbortController` aborted in cleanup, so a stale response can't overwrite a newer one. You also need loading and error states and to check `response.ok`. In production I'd rather use a data library like TanStack Query or framework loaders, which add caching, deduplication, retries and avoid waterfalls."

TL;DR

Fetch-in-Effect needs cleanup (ignore flag or `AbortController`) plus loading/error states. For real apps, prefer TanStack Query or framework loaders.

5.8 `useRef` for values

What it is

```
const ref = useRef(initialValue);   // -> { current: initialValue }, the same object on every render
```

A **ref** is a plain object with a single `current` property that React keeps for the component's lifetime. You can read and change `ref.current` freely, and **changing it does not trigger a re-render**.

Analogy: state is the scoreboard everyone watches — change it and the stadium updates. A ref is a note in the coach's pocket — useful to the coach, invisible to the crowd.

	`useState`	`useRef`
Returns	`[value, setValue]`	`{ current: value }`
Changing it	Via setter → **re-render**	Assign `ref.current = x` → **no re-render**
Value during a render	A fixed snapshot	Mutable; could change at any time
Read it during render?	Yes	**No** (except lazy initialisation)
Use for	Anything shown on screen	IDs of timers, DOM nodes, latest-value holders, instances (created objects) of non-React objects

How it works

`code/06_use_ref_values.jsx`:

```jsx
// 1. Ref vs state: changing a ref never re-renders; the screen shows a stale value until something else renders.
function Compare() {
  const [stateClicks, setStateClicks] = useState(0);
  const refClicks = useRef(0);
  return (
    <>
      <button onClick={() => { refClicks.current++; console.log('  ref is now', refClicks.current); }}>ref
+1</button>
      <button onClick={() => setStateClicks(stateClicks + 1)}>state +1</button>
      <p>state: {stateClicks} | ref (as of last render): {refClicks.current}</p>   {/* reading in render: a bug,
shown on purpose */}
    </>
  );
}

// 2. Stopwatch: the interval ID is needed by a later event but never shown -> ref.
function Stopwatch() {
  const [startTime, setStartTime] = useState(null);
  const [now, setNow] = useState(null);
  const intervalRef = useRef(null);
  function handleStart() {
    setStartTime(Date.now()); setNow(Date.now());
    clearInterval(intervalRef.current);
    intervalRef.current = setInterval(() => setNow(Date.now()), 10);
  }
  function handleStop() { clearInterval(intervalRef.current); }
  const secondsPassed = startTime != null && now != null ? (now - startTime) / 1000 : 0;
  // ... <output>{secondsPassed.toFixed(3)} s</output> + Start / Stop buttons
}

// 3. Lazy-initialise an expensive object once (the only OK "write during render" pattern).
function Player() {
  const engine = useRef(null);
  if (engine.current === null) engine.current = new VideoPlayerEngine();   // runs on the first render only
  // ...
}
```

Output:

```
  ref is now 1
  ref is now 2
after 2 ref clicks  : state: 0 | ref (as of last render): 0
after 1 state click : state: 1 | ref (as of last render): 2
stopwatch ran        : true | still frozen 50 ms after Stop: true
engines created over 4 renders: 1
```

1. The ref really was 2, but the screen said 0 until an unrelated state change re-rendered. That is why **anything displayed must be state**.
2. The stopwatch keeps the interval ID in a ref: "Stop" (a later event) needs it, the UI doesn't. Storing it in state would cause a pointless re-render.
3. `useRef(new VideoPlayerEngine())` would construct an engine on **every** render and throw it away; the `if (ref.current === null)` pattern creates it once. (For state, `useState(() => …)` does the same job — Chapter 3 §3.2.)

Rules

- **Don't read or write `ref.current` during rendering** (except the lazy-init pattern). Rendering must be pure; a value React can't track makes output unpredictable — and the React Compiler (a build tool that automatically memoises components; Chapter 10) assumes you follow this rule.
- Read and write refs in **event handlers and Effects**.
- A ref is **per component instance**, like state. A module-level variable would be shared by all instances.
- The "`usePrevious`" hook you may see online (writing the previous value into a ref in an Effect and reading it during render) breaks the rule above; store the previous value in state as in §5.6 case 5 instead.

In the interview

> **30-second answer:** "`useRef` returns a stable `{ current }` object that persists across renders; mutating it doesn't re-render. I use it for values the UI doesn't display — timer IDs, DOM nodes, instances of non-React libraries, the latest value for a callback — and I only touch it in handlers and Effects, not during render. Anything shown on screen belongs in state."

Self-check

1. Why did the screen show `ref: 0` after two clicks? → Changing a ref doesn't re-render.
2. Where should a `setInterval` ID live if only "Stop" needs it? → A ref.
3. Is `const r = useRef(expensive())` efficient? → No, `expensive()` runs every render; lazy-init with `if (r.current === null)`.

TL;DR

`useRef` = a persistent box whose changes don't re-render. Use it for non-visual values; use state for anything on screen; don't touch refs during render.

5.9 Refs to DOM nodes

What it is

Pass a ref object to a built-in element — `<input ref={inputRef} />` — and React puts the **DOM node** in `inputRef.current` after it creates the element (and sets it back to `null` when the node is removed). Then you can call browser APIs React doesn't wrap: `focus()`, `scrollIntoView()`, `getBoundingClientRect()`, `play()`/`pause()` on media, `showModal()` on `<dialog>`.

How it works

`code/07_dom_refs.jsx`, part 1 and 2:

```jsx
// 1. The basic pattern: useRef(null) + ref={...}; React fills .current after the DOM node exists.
function FocusForm() {
  const inputRef = useRef(null);
  console.log('  during render, inputRef.current =', inputRef.current?.tagName ?? null);
  return (
    <>
      <input ref={inputRef} aria-label="name" />
      <button onClick={() => inputRef.current.focus()}>Focus the input</button>
    </>
  );
}

// 2. A list of refs: one ref holding a Map, filled by ref CALLBACKS. React 19: a callback may return a cleanup.
function CatGallery({ cats }) {
  const itemsRef = useRef(new Map());
  return (
    <>
      <nav>{cats.map((c) => <button key={c} onClick={() => itemsRef.current.get(c).scrollIntoView({ behavior:
'smooth' })}>{c}</button>)}</nav>
      <ul>
        {cats.map((c) => (
          <li key={c}>
            <img alt={c} ref={(node) => {
              itemsRef.current.set(c, node);            // called with the node when attached
              return () => itemsRef.current.delete(c);  // cleanup when detached (React 19+)
            }} />
          </li>
        ))}
      </ul>
    </>
  );
}
```

Output:

```
1 focus:
  during render, inputRef.current = null
  focused element: name
2 list of refs:
  scrollIntoView -> Jellylorum
  after removing Jellylorum, the Map holds: Tom, Maru
```

- During the first render the ref is still `null` — the DOM node doesn't exist yet. React sets refs during the **commit**, before layout Effects (Effects that run before the browser paints, §5.11) run. Use refs in handlers and Effects.
- You can't call `useRef` inside a loop (Hook rules), so for a list use **one ref holding a `Map`** and a **ref callback** per item.

Ref callbacks

A **ref callback** is a function passed as `ref`. React calls it with the DOM node when the element is attached. In **React 19** it can **return a cleanup function**, called when the node is detached — the example removes the cat from the Map. (Before React 19, and still if you return nothing, React calls the callback again with `null` on detach.) Two details:

- An inline callback is a new function each render, so React detaches (runs cleanup) and re-attaches it after every render. That's harmless for simple bookkeeping; wrap it in `useCallback` if the work is expensive.
- Arrow functions with an **implicit return** (no braces, so the expression's value is returned), like `ref={(n) => (instance = n)}`, now look like "returning a cleanup"; TypeScript (typed JavaScript) rejects them. Use a block body `{ instance = n; }`.

When to use DOM refs — and when not

Good uses	Avoid
Focus management (modals, forms, "press / to search")	Changing text, classes, children by hand — let React render them
Scrolling an item into view	Removing nodes React created (React will crash or be confused)
Measuring size/position (§5.11)	Reading input values you could keep in state/`FormData` (unless uncontrolled, Chapter 4 §4.2)
Integrating non-React libraries (charts, maps, editors)	
Media: `play()`, `pause()`; `<dialog>`: `showModal()`	

Tip: to scroll to an item you *just* added with `setState`, the DOM doesn't have it yet inside the handler. Wrap the update in `flushSync(() => setItems(…))` from `react-dom` to commit synchronously, then scroll (Chapter 3 §3.4).

TL;DR

`ref={myRef}` → `myRef.current` is the DOM node after commit (`null` during the first render). Use ref callbacks + a `Map` for lists; React 19 callbacks may return a cleanup. Don't edit DOM that React manages.

5.10 Passing refs to your own components

What it is

By default a component's DOM nodes are private. When a parent legitimately needs a child's node — a design-system (a team's shared kit of UI components) `<TextField>` that the page wants to focus — the child must **pass the ref through**.

- **React 19**: `ref` is an **ordinary prop** for function components. Destructure it and put it on the element.
- **React 18 and earlier**: you had to wrap the component in **`forwardRef`**, which passes `ref` as a second argument. It still works in React 19, but React has announced it will be deprecated; new code should use the prop. (Class components still don't receive `ref` as a prop.)
- **`useImperativeHandle(ref, () => api, deps)`** lets the child expose a **custom, limited object** instead of the raw DOM node — e.g. only `focus()` and `clear()`.

How it works

`code/07_dom_refs.jsx`, parts 3–5:

```jsx
// 3. React 19: `ref` is an ordinary prop for function components — no forwardRef needed.
function TextField({ label, ref }) {
  return <label>{label} <input ref={ref} /></label>;
}
// Legacy (React 18 and earlier) — still works in 19, but forwardRef is planned for deprecation.
const OldTextField = forwardRef(function OldTextField({ label }, ref) {
  return <label>{label} <input ref={ref} /></label>;
});

// 4. useImperativeHandle: expose a small, deliberate API instead of the raw DOM node.
function VideoLikeInput({ ref }) {
  const real = useRef(null);
  useImperativeHandle(ref, () => ({
    focus: () => real.current.focus(),
    clear: () => { real.current.value = ''; },
  }), []);
  return <input ref={real} aria-label="limited" defaultValue="draft" />;
}

// 5. React 19.3: a ref on a <Fragment> gives a FragmentInstance that acts on its children without a wrapper
<div>.
function Toolbar() {
  const group = useRef(null);
  return (
    <>
      <Fragment ref={group}>
        <button>Bold</button><button>Italic</button>
      </Fragment>
      <button onClick={() => group.current.focus()}>focus toolbar</button>
    </>
  );
}
```

Output:

```
3 ref as a prop:
  new-style ref -> INPUT | old-style ref -> INPUT
  focused: New
4 useImperativeHandle:
  exposed keys: [ 'focus', 'clear' ]
  value now "" | focused: limited
5 Fragment ref:
  focused: Bold
```

- Both styles delivered the `<input>` to the parent; the React 19 version is just a prop.
- With `useImperativeHandle` the parent could only `focus` and `clear` — it can't restyle or remove the node. This is good **encapsulation** (hiding internals and exposing only a deliberate interface) for component libraries.
- **Fragment refs** (stable in React 19.3) give a `FragmentInstance` with methods such as `focus()` (focuses the first focusable child), `addEventListener`, `observeUsing(observer)` and `getClientRects()` — useful when you'd otherwise add a wrapper `<div>` only to hold a ref.

Migration

Old	New (React 19)
`const X = forwardRef((props, ref) => …)`	`function X({ ref, ...props }) { … }`
`X.displayName` needed for DevTools with anonymous `forwardRef`	Normal named function
`<Ctx.Provider value>` (related 19 change)	`<Ctx value>` (Chapter 6 §6.3)

React provides a **codemod** (a script that rewrites your source code automatically) for the `forwardRef` migration.

In the interview

> **30-second answer:** "To give a parent access to a child's DOM node, in React 19 the child just accepts `ref` as a prop and passes it to the element; `forwardRef` is the older way and is on the path to deprecation. If I don't want to expose the whole node, `useImperativeHandle` exposes a small API like `focus`. Refs are an escape hatch — I use them for focus, scroll, measurement and third-party widgets, not for rendering."

TL;DR

React 19: `function Field({ ref }) { return <input ref={ref} /> }`. `forwardRef` = legacy. `useImperativeHandle` exposes a limited API. React 19.3 adds refs on `<Fragment>`.

5.11 `useLayoutEffect` — before the browser paints

What it is

`useLayoutEffect` has the same signature as `useEffect`, but it runs **synchronously after React updates the DOM and before the browser paints**. The browser waits for it. That makes it the right tool for one job: **measure layout and adjust before the user sees anything** — e.g. position a **tooltip** (a small label that pops up next to an element), or scroll a chat to the bottom without a visible jump.

`useInsertionEffect` runs even earlier (before layout effects) and exists for CSS-in-JS libraries (libraries where you write styles inside JavaScript, Chapter 8) to insert `<style>` tags; app code rarely needs it.

How it works

`code/08_layout_effect.jsx` — first the order of work, then a tooltip that must flip below its target when there's no room above:

```
function Order() {
  const [n, setN] = useState(0);
  console.log(`  render (n=${n})`);
  useInsertionEffect(() => { console.log('  useInsertionEffect  (for CSS-in-JS libraries: inject <style> first)');
});
  useLayoutEffect(() => { console.log('  useLayoutEffect     (DOM updated, not painted yet: measure here)');
return () => console.log('  layout cleanup'); });
  useEffect(() => { console.log('  useEffect           (after paint: subscriptions, fetching, logging)'); return
() => console.log('  effect cleanup'); });
  return <button onClick={() => setN(n + 1)}>n={n}</button>;
}

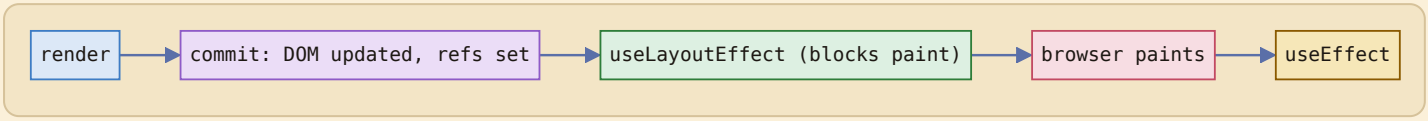
function Tooltip({ targetTop, useLayout }) {
  const ref = useRef(null);
  const [height, setHeight] = useState(0);                    // unknown until measured
  const measure = () => setHeight(ref.current.getBoundingClientRect().height);
  (useLayout ? useLayoutEffect : useEffect)(measure, []);
  const placeAbove = targetTop - height >= 0;                  // enough room above the target?
  const top = placeAbove ? targetTop - height : targetTop + 30;
  return <div ref={ref} data-h="80" role="tooltip" style={{ position: 'absolute', top }}>{placeAbove ? 'above' :
'below'} the button</div>;
}
// Rendered with createRoot + flushSync like a browser task; "first paint" = the DOM at the end of that task.
```

Output:

```
mount:
  render (n=0)
  useInsertionEffect  (for CSS-in-JS libraries: inject <style> first)
  useLayoutEffect     (DOM updated, not painted yet: measure here)
  useEffect           (after paint: subscriptions, fetching, logging)
click:
  render (n=1)
  useInsertionEffect  (for CSS-in-JS libraries: inject <style> first)
  layout cleanup
  useLayoutEffect     (DOM updated, not painted yet: measure here)
  effect cleanup
  useEffect           (after paint: subscriptions, fetching, logging)
  layout cleanup
  effect cleanup
useEffect      : first paint "above the button" top=50px | settled "below the button" top=80px
useLayoutEffect: first paint "below the button" top=80px | settled "below the button" top=80px
```

(The last two "cleanup" lines are the unmount between demos. The tooltip's 80 px height is faked because jsdom has no layout.)

- The tooltip can't know its height until it's in the DOM, so the first render guesses (height 0 → "above").
- With `useEffect`, that wrong first position reaches the screen; the fix arrives a frame later — a visible **flicker**.
- With `useLayoutEffect`, React measures and re-renders **before** the browser paints, so the user only ever sees the correct position.



Trade-offs

useEffect	useLayoutEffect
Doesn't block painting — **default choice**	Blocks painting — slow code here makes the app feel janky (stuttering, not smooth)
May show one frame of the "before" state	No flicker for measure-then-adjust
Fine for 99% of Effects	Only for reading layout and synchronously fixing it
—	Does nothing during server rendering (the HTML is sent before any layout exists)

TL;DR

`useLayoutEffect` = runs after DOM update, before paint; use it to measure and reposition without flicker. Everything else: `useEffect`.

5.12 Putting it together — a city search

`code/citysearch.jsx` is a small search page that uses each tool for its proper job:

Need	Tool	Why
Debounce the query (wait until typing pauses)	Effect with `setTimeout` + cleanup (in a tiny `useDebounced` Hook — custom Hooks, your own reusable `use…` functions, are Chapter 7)	Timer = external system; each keystroke's cleanup cancels the previous timer
Focus the box when the page opens	DOM ref + Effect	Focus is a browser API
Press `/` anywhere to focus search	Window listener in an Effect + cleanup	Global subscription while mounted
Search the (fake) API	Effect with `AbortController`	Server sync, race-safe
Keep the tab title in sync	Effect writing `document.title`	Browser API
Tooltip that flips if there's no room	Refs + `useLayoutEffect`	Measure before paint
Result count	Plain variable	Derived — no Effect

Key parts:

```jsx
// Effect 1 (timer): a debounced copy of a value — only updates after `delay` ms without changes.
function useDebounced(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);                    // a new keystroke cancels the pending update
  }, [value, delay]);
  return debounced;
}

export default function CitySearch({ searchCities, delay = 250, initialQuery = '', tipFor = null }) {
  const [query, setQuery] = useState(initialQuery);
  const [result, setResult] = useState({ status: 'idle', cities: [] });
  const inputRef = useRef(null);
  const debouncedQuery = useDebounced(query.trim(), delay);

  // Effect 2 (DOM): focus the box when the page opens.
  useEffect(() => { inputRef.current.focus(); }, []);

  // Effect 3 (global listener): press "/" anywhere to jump to the search box.
  useEffect(() => {
    function onKeyDown(e) {
      if (e.key === '/' && document.activeElement !== inputRef.current) { e.preventDefault();
inputRef.current.focus(); }
    }
    window.addEventListener('keydown', onKeyDown);
    return () => window.removeEventListener('keydown', onKeyDown);
  }, []);

  // Effect 4 (network): fetch for the debounced query; abort the previous request on change.
  useEffect(() => {
    if (!debouncedQuery) { setResult({ status: 'idle', cities: [] }); return; }
    const controller = new AbortController();
    setResult((r) => ({ ...r, status: 'loading' }));  // keep old results visible while loading
    searchCities(debouncedQuery, { signal: controller.signal })
      .then((cities) => setResult({ status: 'done', cities }))
      .catch((e) => { if (e.name !== 'AbortError') setResult({ status: 'error', cities: [], message: e.message });
});
    return () => controller.abort();
  }, [debouncedQuery, searchCities]);

  // Effect 5 (browser API): keep the tab title in sync.
  useEffect(() => {
    document.title = debouncedQuery ? `“${debouncedQuery}” – City search` : 'City search';
  }, [debouncedQuery]);

  const count = result.cities.length;                 // derived
  // ... input, status line, <CityRow> list
}

// In CityRow: measure the tooltip before paint and flip it below the row if it would overflow the top.
// (It lives in the row, not in the tooltip: a parent's layout effect runs after the refs of its children are
attached.)
useLayoutEffect(() => {
  if (!hover) { setPos(null); return; }
  const row = rowRef.current.getBoundingClientRect();
  const tipHeight = tipRef.current.getBoundingClientRect().height;
  const below = row.top - tipHeight - 6 < 0;          // no room above -> show below
  setPos({ top: (below ? row.bottom + 6 : row.top - tipHeight - 6) + window.scrollY, left: row.left + 180, below
});
}, [hover]);
```

`code/09_city_search.jsx` drives it (debounce 50 ms; the fake API is slow for short queries so a race can happen):

Output:

```
on open, focused: true | Start typing to search.
typed "osa"  -> Searching… | requests so far: [ 'osa' ]
             -> 1 city for “osa” [ 'Osaka' ] | title: “osa” – City search
"to" then "tor" -> 1 city for “tor” [ 'Toronto' ] | requests: [ 'to', 'aborted "to"', 'tor' ]
"error"      -> Error: Service unavailable
pressed "/"  -> focused: true
```

- Typing "o", "s", "a" produced **one** request, for "osa" — the debounce cleanups cancelled the earlier timers.
- When the user typed "r" while the "to" request was still in flight, the Effect's cleanup **aborted** it; only "tor" results were shown.
- Errors became a visible state; aborts didn't.
- The `/` shortcut worked because the listener was registered while the component was mounted.

In the browser (`code/citysearch.html`, styles in `citysearch.css`; the entry opens with the query "o" and Tokyo's tooltip shown):

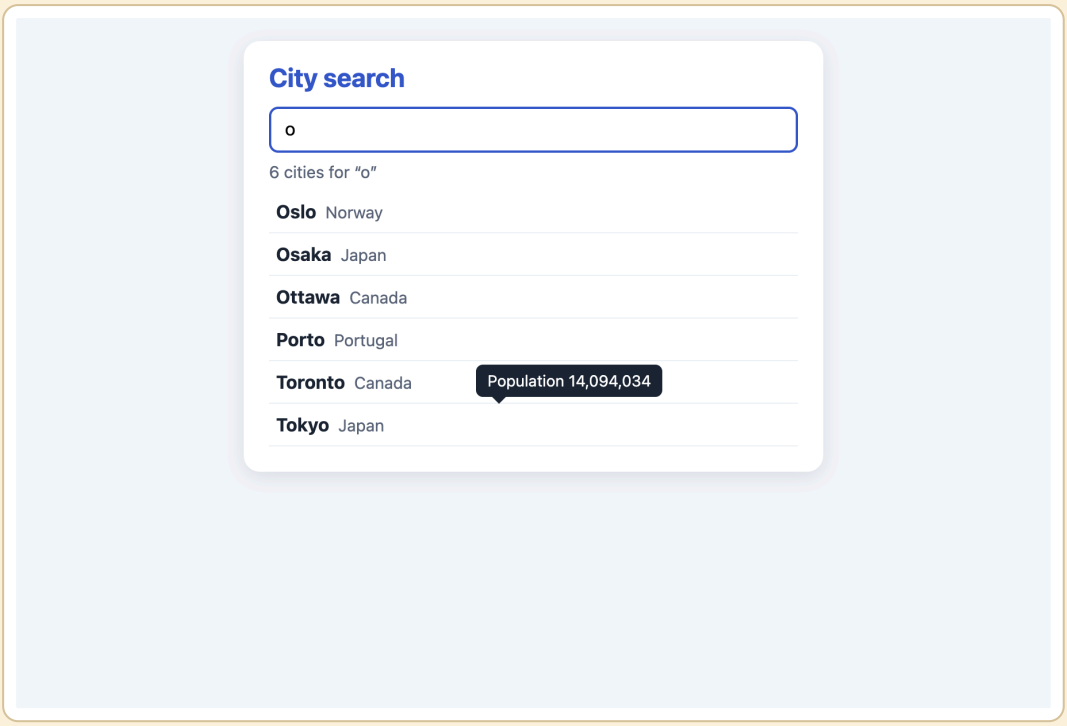


Figure 5.1 — The city search. The focus came from a ref + Effect, the list from an aborted-when-stale fetch Effect, and the tooltip's position from `useLayoutEffect` measuring before paint.

Design notes:

- **Every Effect names its external system** (timer, DOM focus, window events, network, document title). Nothing merely copies props into state.
- **Every Effect cleans up**, so StrictMode's double run and fast typing are both harmless.
- In a production app, Effect 4 would be one line with TanStack Query (`useQuery({ queryKey: ['cities', q], queryFn })` — Chapter 6 §6.9), which also caches results.

Chapter 5 TL;DR

Idea	Remember
Where code goes	Pure calc → render; user action → handler; sync with outside while on screen → Effect
`useEffect`	Runs after commit; `[]` = mount only; `[a]` = when `a` changes; no array = every render
Cleanup	Return a function that undoes setup: disconnect, clear, remove, abort
StrictMode	Dev-only setup → cleanup → setup; fix cleanups, don't suppress
Dependencies	Everything reactive the Effect reads; reduce by moving objects inside, updaters, splitting, `useEffectEvent`
Not needed	Derived data, resets (`key`), event logic, notifying parents, chains, app init
Fetching	Ignore flag or `AbortController`; loading/error; prefer TanStack Query / loaders
`useRef`	`{ current }`, persists, no re-render; not for displayed data; not read in render
DOM refs	`ref={r}` → node after commit; ref callbacks (+ cleanup in 19) for lists
Refs to components	React 19 `ref` prop; `forwardRef` legacy; `useImperativeHandle` for limited APIs; Fragment refs in 19.3
`useLayoutEffect`	Before paint; measure and adjust without flicker; otherwise use `useEffect`

Next: Chapter 6 — State Management.